

Week-3 Day-5

Project Overview

This was the Friday hackathon task for building an analytics pipeline on top of a large AdventureWorks-style PostgreSQL database.

The main idea was not to keep writing random isolated SQL queries directly on raw operational tables. The goal was to create a reusable analytics layer first, and then use that analytics layer for dashboard-style analysis in Python.

So the flow I followed was:

- raw PostgreSQL tables
- reusable analytics views
- business metric views
- dashboard-ready views
- pandas notebook visualizations
- executive recommendations

Database Overview

The database is split into multiple schemas, which was confusing at first because most previous databases I used were just one schema with some tables.

The important schemas used in this project were:

- `sales`
- `production`
- `person`
- `humanresources`
- `purchasing`

The main business areas covered were:

- Sales and orders
- Products and categories
- Customers
- Territories
- Salespeople / employees
- Inventory
- Vendors and purchasing

Analytics Architecture

I created an `analytics` schema and used it as the reporting layer.

The most important base view is:

- `analytics.sales_order_lines`

This view joins order headers, order details, products, product categories, and territories. It creates the reusable line-level sales dataset. A “line” here means one product row inside one order.

After that, the other views reuse this base view instead of recalculating joins again and again.

Views Created

Task 1 required at least 10 reusable analytical views. These are the main 10:

1. `analytics.sales_order_lines`
2. `analytics.monthly_sales`
3. `analytics.product_performance`
4. `analytics.category_performance`
5. `analytics.quarterly_sales`
6. `analytics.territory_performance`
7. `analytics.customer_performance`
8. `analytics.salesperson_performance`
9. `analytics.inventory_health`
10. `analytics.vendor_purchasing_performance`

Then I added 4 more dashboard/executive views for Tasks 2, 3, and 4:

1. `analytics.monthly_sales_growth`
2. `analytics.product_rankings`
3. `analytics.territory_rankings`
4. `analytics.executive_kpi_summary`

These final views are the ones that make the Python notebook easier because pandas can read clean final datasets instead of joining raw tables.

SQL Design Decisions

The main design decision was to build from a base analytics view instead of repeating the same joins everywhere.

For example:

- `sales_order_lines` calculates revenue, cost, and profit at the order line level.
- `monthly_sales` groups that data by month.
- `product_performance` groups that data by product.
- `territory_performance` groups that data by territory.
- the ranking and KPI views build on those already aggregated views.

I used `LEFT JOIN` where I only needed descriptive lookup data, like product category names or territory names. This avoids accidentally removing rows if some lookup value is missing.

I also used:

- `CASE WHEN` for customer segmentation and inventory status
- `COALESCE` where a missing sales value should show as 0
- CTEs for monthly growth and executive KPI summaries
- window functions like `LAG()` for growth
- ranking functions like `RANK()` for product and territory rankings
- conditional aggregation with `SUM(CASE WHEN ... THEN 1 ELSE 0 END)`

Python Notebook

The notebook is:

- `executive_analysis.ipynb`

In the notebook, I connected to PostgreSQL using:

- `pandas`
- `SQLAlchemy`
- `psycopg`

The notebook only reads from the final `analytics` views. It does not query raw operational tables.

The charts created were:

1. Monthly revenue trend
2. Month-over-month revenue growth
3. Revenue by territory
4. Customer segments
5. Top products by revenue
6. Revenue by product category
7. Top salespeople by revenue
8. Inventory health status

The chart PNG files are saved in:

- `charts/`

Key Results

- Total revenue was about **\$109.8M**.
- Total profit was about **\$9.37M**.
- Bikes generated about **\$94.7M** revenue, which is far higher than every other category.
- Southwest was the top territory by revenue, with about **\$24.2M** revenue.
- Australia was the top territory by profit, with about **\$3.4M** profit.
- High Value customers were a small group, only **244 customers**, but they generated about **\$71.5M** revenue.
- There were **80 Low Stock** products and **61 Watch** products.

Business Opportunities

1. **Focus more on Bikes.**
Bikes are clearly the main business driver and generated most of the revenue.
2. **Use Southwest as a strong territory benchmark.**
Southwest had the highest revenue, so it is worth studying what is working there.
3. **Protect High Value customers.**
A small number of High Value customers generated a huge amount of revenue.
4. **Promote strong Mountain bike products.**
The top products by revenue were mostly Mountain bikes.
5. **Study Australia for profitability.**
Australia was not number 1 in revenue, but it was number 1 in profit.

Business Risks

1. **Revenue depends heavily on Bikes.**
If bike demand drops, the whole business can be affected badly.
2. **Some territories have negative profit.**
Northeast, Southeast, and Central need investigation.
3. **Low stock products can cause missed sales.**
Low Stock and Watch products should be reviewed.
4. **Most customers are Low Value.**
The company should try to move some customers into higher value segments.
5. **Some high revenue products are weak in profit.**
Revenue alone is not enough for decision making.

Recommendations

1. Build retention campaigns for High Value customers.
2. Review negative-profit territories and understand the cost or discounting issue.
3. Prioritize inventory checks for Low Stock products.
4. Keep Mountain bikes as a core sales and inventory focus.
5. Use profit ranking along with revenue ranking in dashboards.

Challenges Faced

The biggest challenge was simply understanding the database. The database has multiple schemas and many tables, so it was hard at first to know where the

important relationships were.

Another challenge was the original CSV dataset. Some files did not load correctly at first because of formatting/delimiter issues. After the corrected dataset was provided, the database loaded properly and all tables had data.

Another challenge was understanding the difference between order header and order detail. The final understanding was:

- order header = one row for the whole order
- order detail = product rows inside that order

This became important because most useful sales analytics needed to happen at the order line level.

Assumptions Made

- Revenue was calculated as `orderqty * unitprice * (1 - unitpricediscount)`.
- Cost was calculated as `orderqty * standardcost`.
- Profit was calculated as revenue minus cost.
- Customer segments were based on total revenue:
 - High Value: revenue ≥ 100000
 - Medium Value: revenue ≥ 10000
 - Low Value: revenue < 10000
- Inventory status was based on `quantity_on_hand`, `reorderpoint`, and `safetystocklevel`.

Deliverables

- `analytics_pipelines.sql` contains the full SQL analytics pipeline.
- `executive_analysis.ipynb` contains the PostgreSQL connection, pandas reads, charts, insights, and recommendations.
- `charts/` contains the saved chart images.
- `screenshots/` contains screenshots of SQL outputs.
- `documentation/` contains the PDF version of this README.